Programmable Data Plane

Programmable Networks A.Y. 2024/25

Outline

- Introduction
- The programmable data plane
- Programmable switch architectures
  - PISA
  - Open State

Outline

- Introduction
- The programmable data plane
- Programmable switch architectures
  - PISA
  - Open State

Introduction

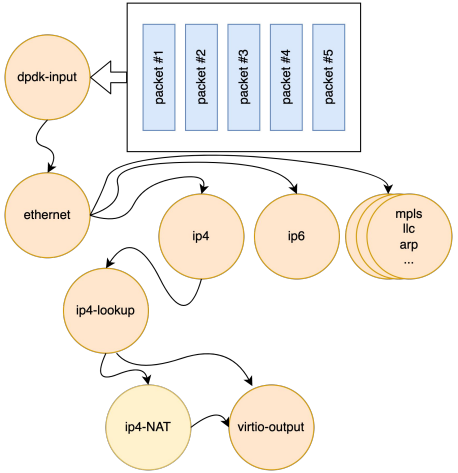
- Network switches are called to perform a restricted set of well identified task:
  - rewrite packet contents
  - encapsulate/decapsulate tunnel headers
  - drop or forward the packet
  - add/remove/update packet tags, labels, or header fields
  - etc.
- Fixed-function switch chips do implement such processing throughout a set of fixed size lookup tables arranged in a pre-determined pipeline order
  - The widths, depths, and execution order are set when the chip is fabricated
- This severely limits flexibility

The quest for programmability

- **Data plane programmability is a win-win paradigm**
  - a new functionality can be quickly implemented and deployed directly by the operator using the device programming interface
  - frees device suppliers from designing network equipment for a wide range of customer use cases
- NFV represents a first incarnation of data plane programmability
  - it exploits traditional programming platforms and languages to define new network functions
  - use of cheap and massively scalable cloud infrastructures allows for CAPEX cost reduction

Softwarization: not a panacea!

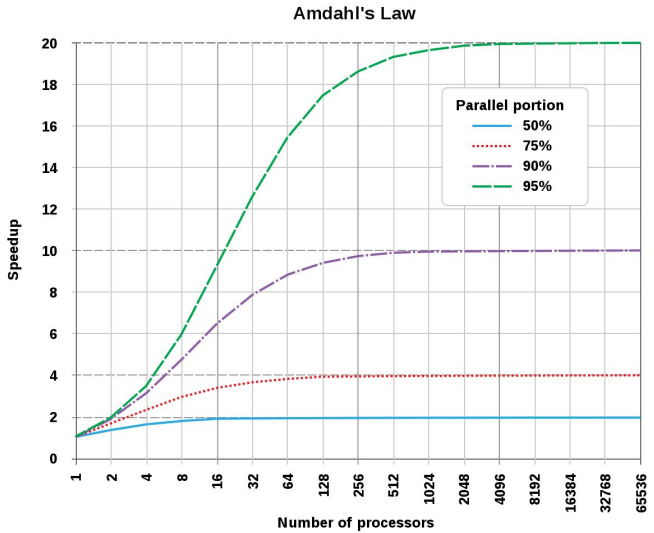
- However, softwarization by itself is not enough
  - performance becomes a bottleneck, especially in productive networks with huge traffic volumes
- The answer of NFV to performance requirements is twofold
  - software acceleration techniques
    - DPDK, VPP, etc.
  - horizontal scaling
    - run multiple instances of a VNF and parallelize the computation



Amdahl's Law

- These solutions are unfortunately only an apparent panacea, which may not work out well in long term, in sight of a very basic principle known as Amdahl's law
  - **F** is the percentage of parallelizability of the task,
  - **N** is the number of processing units available
- Example
  - F = 99% and N = 128 cores, the best we can reach is a 56x speedup
    - **less than half of the available cores!**

$$\text{speedup} = \frac{1}{(1 - F) + F/N}$$



Programmable data planes to the rescue

- Need to address the dichotomy between the flexibility of software and the quest for very high performance levels
- Basic idea
  - rely on ASIC for performance guarantee
    - e.g., Intel Barefoot Tofino
  - add a degree of programmability through a domain specific language
    - e.g., P4 language

Outline

- Introduction
- The programmable data plane
- Programmable switch architectures
  - PISA
  - Open State

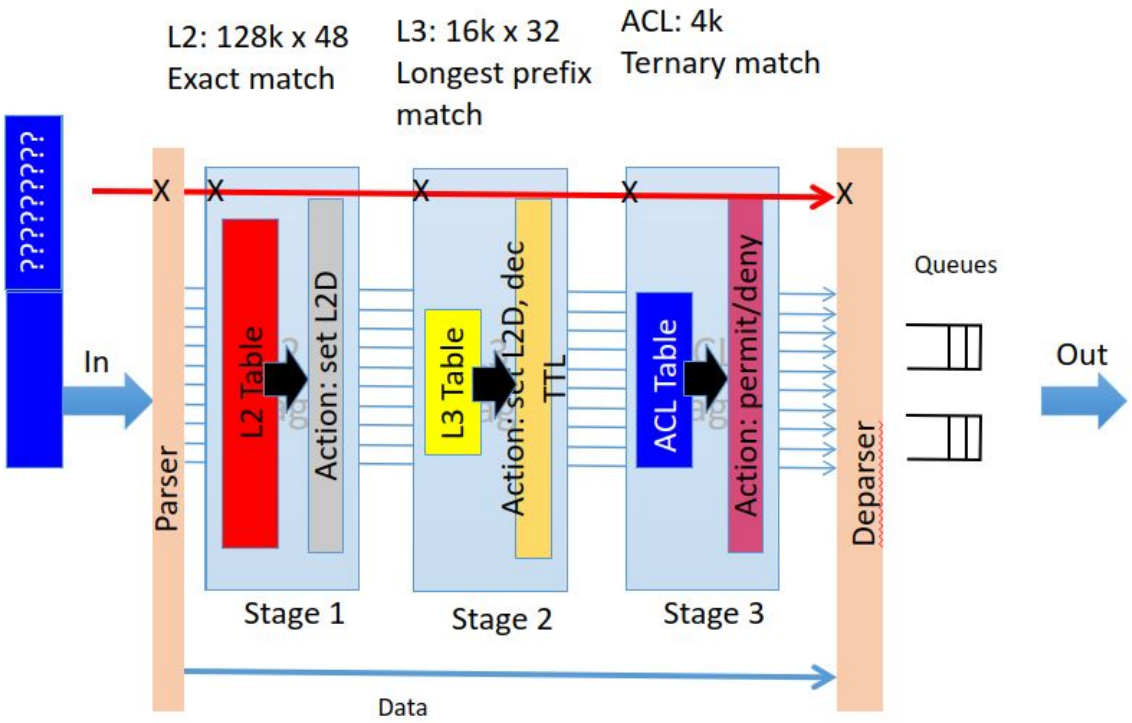
Conventional Data Plane

- In conventional network devices control and the data planes integrated into each device
  - The **control plane** is in charge of establishing packet processing policies
  - The **data plane**, in contrast, is responsible solely for executing the packet processing policy set by the control plane
- To enforce control plane decisions, the data plane performs a set of well defined operations on received packets
  - it **parses** (a subset of) the packet bytes to understand the control information they carry
  - it determine the sequence of **processing operations** that need to be applied to the packet (e.g., checksum calculation, new checksum writing, counter update, etc.)
  - it **forwards** the modified packet based on the results of the performed operations

Match Table

- **Single Match Table (SMT)**
  - All header in one table
  - To support flexibility, the table needs to store all combination in the header
  - Wasteful
- **Multiple Match Tables (MMT)**
  - Smaller tables with a subset of headers in a pipeline of stages
  - Stage j can be made depend on stage i, i<j
  - Existing switches has a small (4-8) number of tables whose widths, depths, and execution order are fixed

An MMT fixed function switch



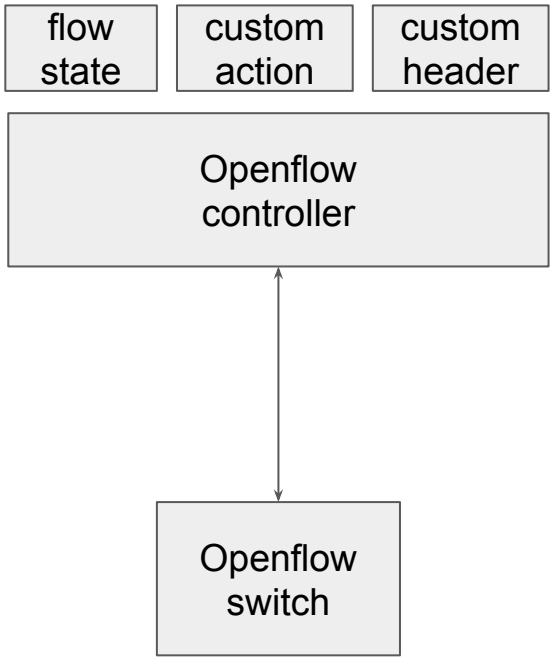
Programmable Data Plane (PDP): a definition

programmability refers to the capability of a switch to expose the packet processing logic to the control plane to be systematically, rapidly, and comprehensively reconfigured

- In programmable switches it is possible to customize
  - the parsing of packet headers
  - provision forwarding tables that match on arbitrary header fields at arbitrary layers
  - introduce new processing actions

Does Openflow meet the flexibility of PDP?

- **Pros:**
  - multi layer support
  - match+action abstraction
    - compatible with switch hardware
    - sufficiently expressive to permit innovative network functions
  - actions set already supported by the hardware
- **Cons:**
  - does not support
    - new actions
    - new protocols
    - stateful processing



Outline

- Introduction
- The programmable data plane
- Programmable switch architectures
  - PISA
  - Open State

Programmable Switch Architectures

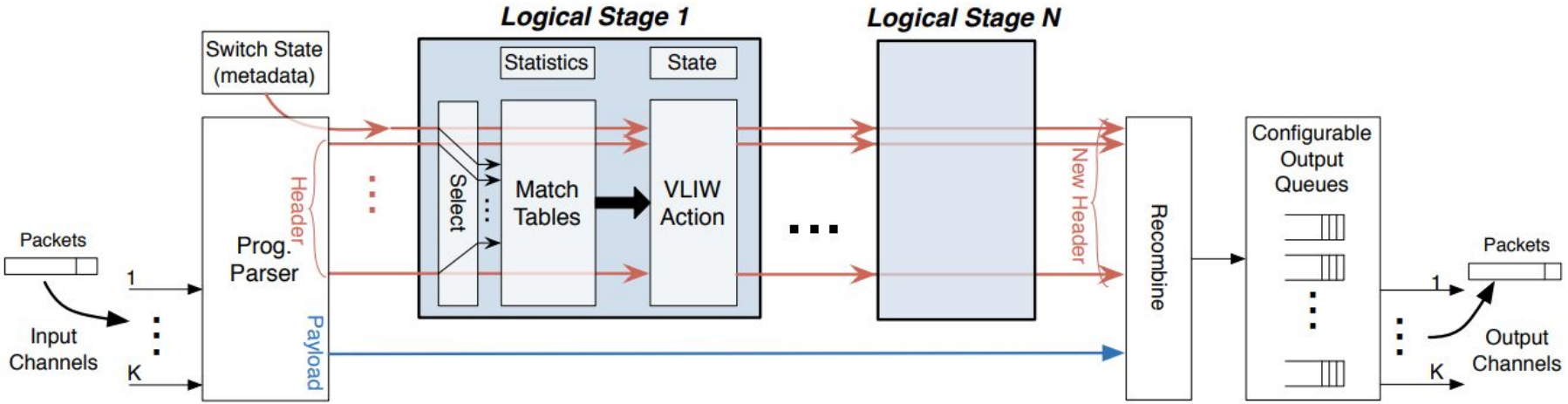
- **Challenge**
  - find the right subset of elementary tasks and composition primitives which permit the right level of flexibility for developing packet/flow processing tasks, while guaranteeing wire speed performance
- **Solution**
  - extend the match/action abstraction, allowing some degree of programmability
- Two common architectures are
  - Protocol Independent Switch Architecture (PISA)
  - Open State

Outline

- Introduction
- The programmable data plane
- Programmable switch architectures
  - PISA
  - Open State

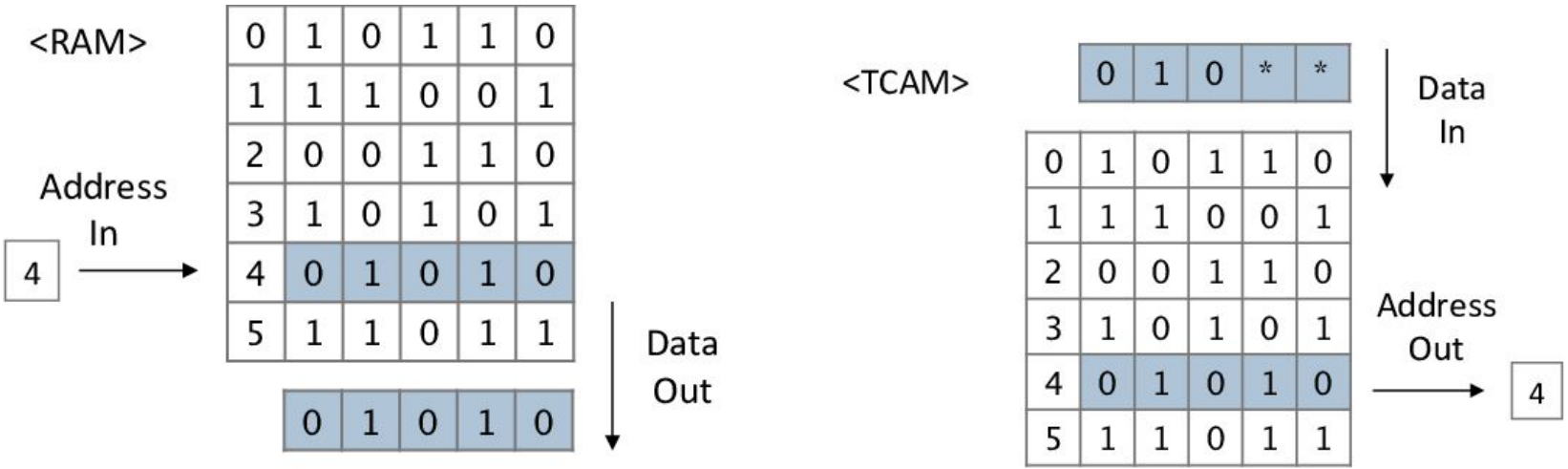
PISA Architecture

- RMT's breakthrough idea is to **intertwine**, in sequence of pipelined Match/Action stages, processing capabilities (deployed via VLIW ALUs)



TCAM vs RAM

- Table lookup is an hardware-friendly abstraction thanks to the usage of Ternary Content Addressable Memories (TCAMs)
  - it retains O(1) lookup time - 2 clock cycles

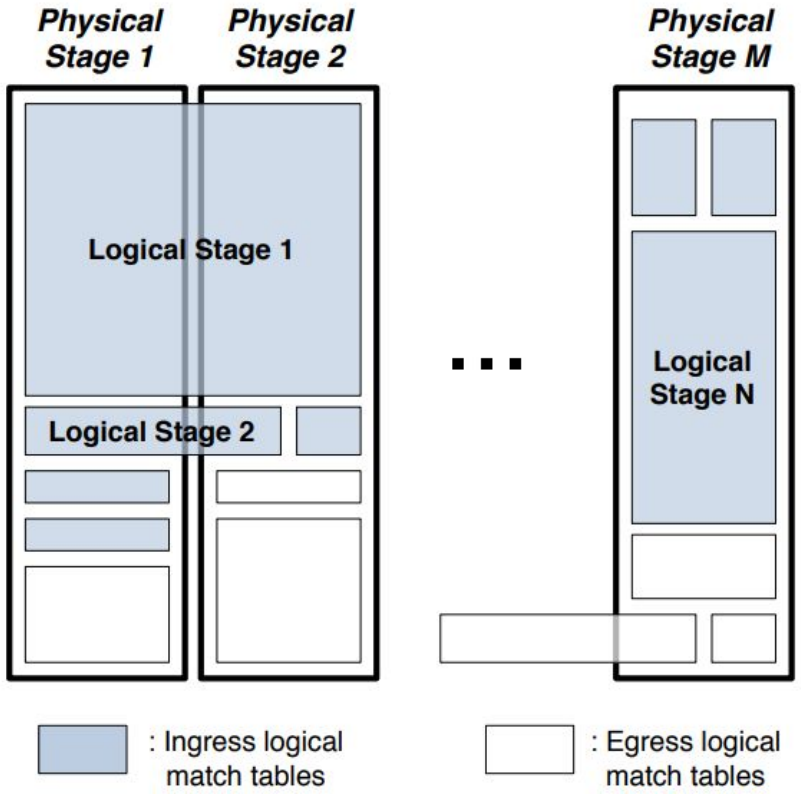


Reconfigurable Match Tables (RMT)

- **RMT** allows a set of pipeline stages, each with a match table of arbitrary depth and width
- RMT goes beyond fixed function MMT in four ways
  - field definitions can be changed and new fields can be added
  - the number, topology, widths and depths of match tables can be specified
  - new actions can be defined
  - arbitrarily modified packets can be placed in specified queue(s)
- The configuration should be done by an SDN controller

Flexible match table configuration

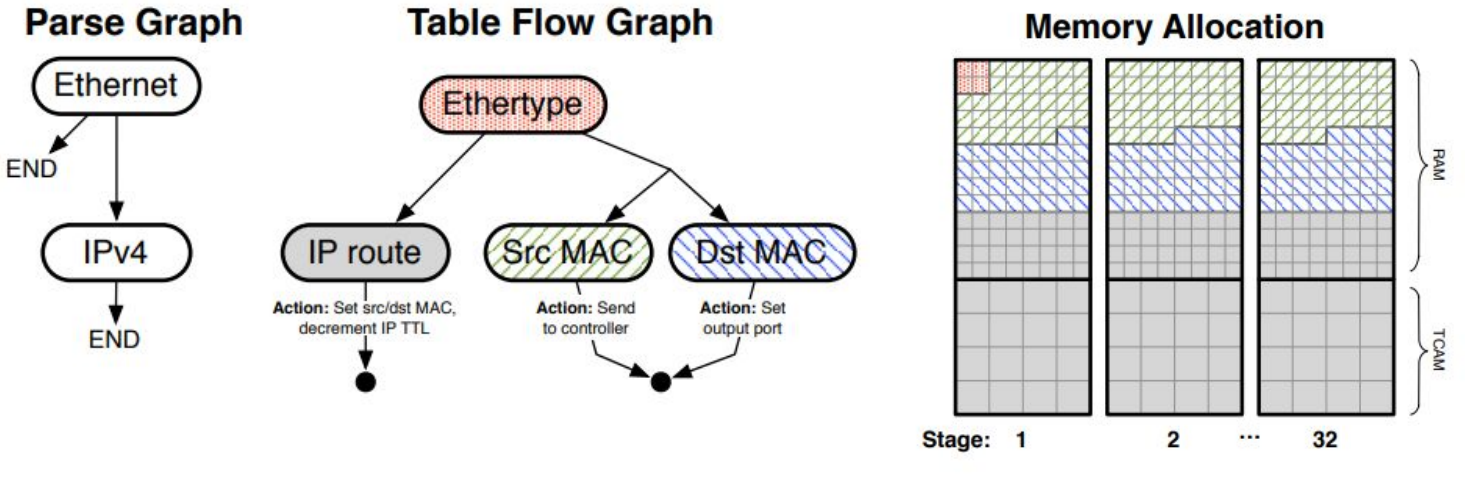
- A physical match stage is represented by a table memory (TCAM or RAM)
- Logical match stages dynamically mapped on physical ones
- **Example**
  - each physical match stage allows only 1,000 prefix entries
    - a 2,000 IP logical match table is implemented in two stages
    - a small Ethertype match table could occupy a small portion of a match stage's memory



Example: L2/L3 switch

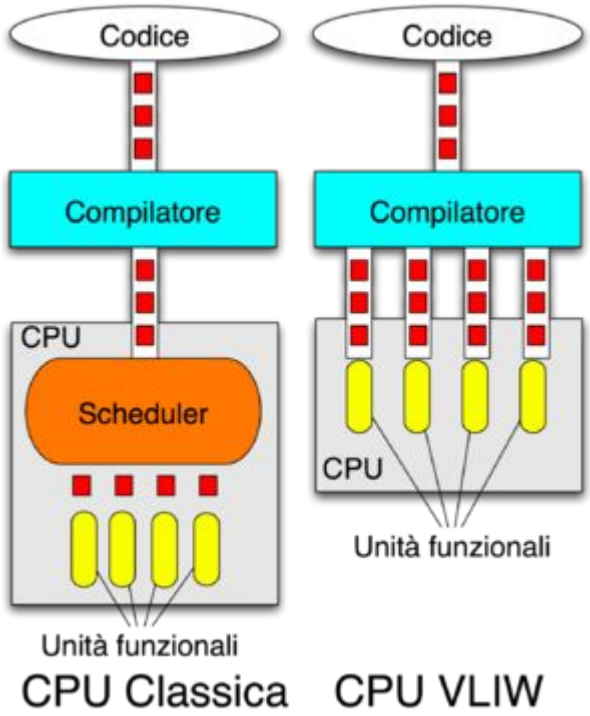
- It is required to define:
  - the **parse graph**
    - tells us the parser to extract and place four fields (Ethertype, IP DA, L2 SA, L2 DA) on the wide header bus
  - the **table flow graph**
    - tells us which fields should be read from the wide header bus and matched in the tables
  - the **memory allocation**
    - tells us how the four logical tables are mapped to the physical memory stages

Example: L2/L3 switch

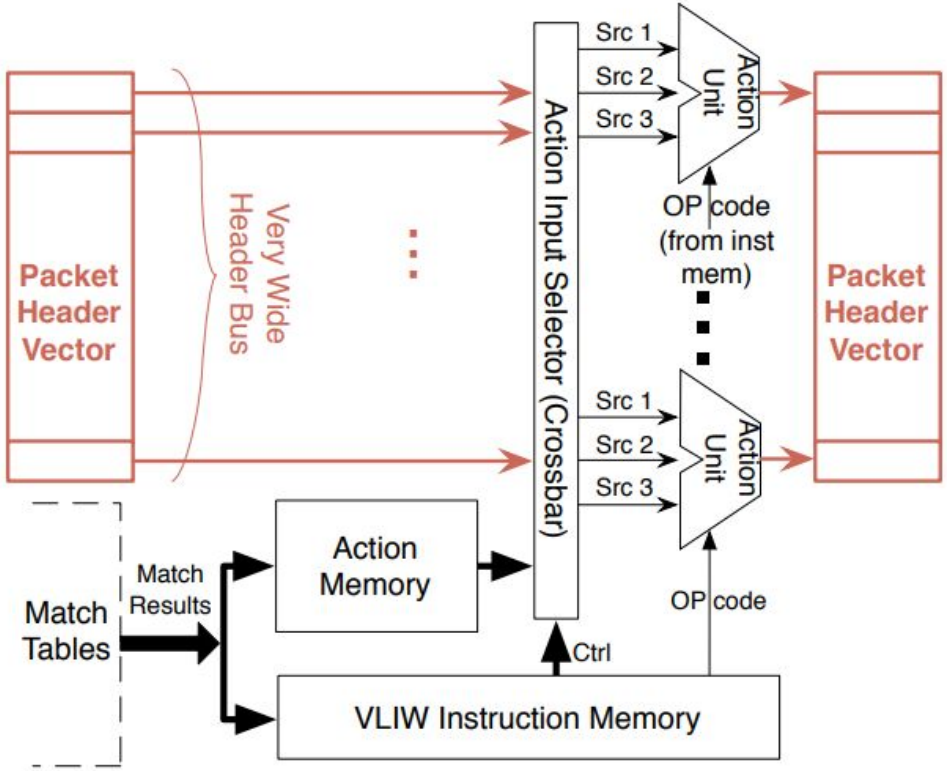


Very Long Instruction Word

- Classical CPUs execute instructions in pipeline
- A **VLIW** processor allows programs to explicitly specify instructions to execute in parallel
- The complexity of determine what instructions can be executed in parallel is offloaded to the compiler



VLIW action architecture



Outline

- Introduction
- The programmable data plane
- Programmable switch architectures
  - PISA
  - Open State

Stateful programmable data plane architecture

- OpenState generalizes the “match-action” abstraction while retaining full compatibility with the OpenFlow hardware
- match/action abstraction can be reinterpreted as a “map” function

$$T : I \longrightarrow O$$

  - $I = \{i_1, \ldots, i_M\}$ is a finite set of **Input Symbols**
    - all the possible matches supported by an OpenFlow specification
  - $O = \{o_1, \ldots, o_K\}$ is a finite set of **Output Symbols**
    - all the possible actions supported by an OpenFlow switch
- the “engine” which performs the mapping is a standard TCAM

Openflow modeled as a Mealy Machine

- Such an OpenFlow “map” abstraction is a very special case of a Mealy Machine (Finite State Machine with output)
- It permits to formally model dynamic forwarding behaviors
  - i.e., to change in time the specific action associated to a same match
- A new mapping can be defined as

$$T : S \times I \longrightarrow S \times O$$

  - where $S = \{s_1, s_2, \dots, s_N\}$ is the **set of programmer-specific states**
- Two key differences with respect to the original OpenFlow abstraction
  - depending on the current state, for the same input, different output are possible
  - state transition is possible

Example: port knocking

- A host that wants to establish an SSH connection toward a server which is behind a firewall
- It delivers a sequence of packets addressed to an ordered list of pre-specified closed ports (e.g., 5123, 6234, 7345 and 8456)
- Once the exact sequence of packets is received, the firewall opens port 22 for the considered host